Instituto Tecnológico de Veracruz

Manual Melodic

Migración de una plataforma de música en streaming de SQL a MongoDB

Integrantes del equipo AlgoFichas:

Luis Daniel Morales Villarauz

Sergio Emanuel Soberano Paredes

Materia: Bases de Datos NoSQL

Profesor: Jose Nahum Prieto Velazquez

20 de octubre de 2026

Índice

1. Introducción **5**
1.1 Objetivo general 5
1.2 Objetivos específicos 5
1.3 Alcance de este documento 6
2. Sistema origen: Melodic **6**
2.1 Descripción del negocio 6
2.2 Problemática que resuelve el sistema 6
2.3 Arquitectura actual 7
2.4 Roles y módulos 7
3. Justificación de la migración a NoSQL **8**
3.1 Por qué MongoDB es adecuado para Melodic 8
3.2 Qué se gana y qué se cede 8
4. Análisis del modelo relacional actual **9**
4.1 Inventario de tablas 9
4.2 Diferencias entre la documentación y el repositorio 11
4.3 Patrones de acceso 11
5. Estrategia de modelado **12**
5.1 Criterios 12
5.2 Decisión principal: playlists con canciones embebidas 13
5.3 Patrones aplicados 13
5.4 Reglas de negocio actualizadas 14
6. Mapeo SQL → MongoDB **14**
6.1 Mapeo de tablas a colecciones 14
6.2 Mapeo campo por campo 15
7. Diagrama de documentos **16**
7.1 Ejemplos por colección 16
Colección users 16
Colección subscriptions 17
Colección albums 17
Colección songs 17
Colección playlists 17
Colección royalties 17
Colección earnings 17
Colección expenses 17
8. Arquitectura de MongoDB **18**
8.1 Componentes 18
8.2 Opciones de despliegue evaluadas 18
9. Diagrama del modelo de datos (colecciones) **19**
9.1 Relaciones entre colecciones 19
10. Índices y validación de esquema **20**

10.1 Índices....20
10.2 Validación de esquema....21
11. Estrategia de migración (ETL)....22
11.1 Fases....22
11.2 Esquema del script....22
11.3 Validaciones posteriores a la carga....22
11.4 Corte y reversión....23
12. Seguridad y hardening....24
12.1 Controles implementados....24
12.2 Hallazgos de seguridad en el sistema original....24
12.3 Seguridad local frente a producción....24
13. Vistas del sistema (frontend existente)....25
14. Implementación y evidencias....26
14.1 Qué se logró....26
14.2 Ramas del repositorio....26
15. Arquitectura final....27
16. Datos y migración....28
16.1 Datos sintéticos....28
16.2 Resultado....28
17. Seguridad: autenticación y control de acceso....29
17.1 En MongoDB....29
17.2 En la aplicación....30
17.3 En los contenedores....30
18. Cifrado en tránsito....31
18.1 Del navegador al servidor....31
18.2 De la aplicación a MongoDB....31
19. Seguridad de red....32
19.1 Firewall....32
19.2 Puertos expuestos....32
19.3 VPN....33
20. Despliegue en línea....34
20.1 Servidor....34
20.2 Pasos....34
20.3 Dominio y variables de entorno....35
20.4 Carga de datos....35
20.5 Cuentas de acceso....36
20.6 Errores corregidos en producción....36
21. Consultas y comandos en MongoDB....37
21.1 Conexión....37
21.2 CRUD....37
21.3 Operaciones clave con los datos reales....38
21.4 Agregaciones....40

21.5 Análisis con explain..... 40
21.6 Comandos de administración..... 42
21.7 Respaldo y restauración..... 43
22. Uso de la plataforma..... 43
22.1 Probar Melodic en línea..... 43
22.2 Instalación en un equipo propio..... 44
22.3 Recorrido guiado por rol..... 45
Como artista..... 45
Como oyente..... 45
Como sello..... 46
Como administrador..... 46
22.4 Comprobaciones..... 46
22.5 Problemas comunes..... 46
23. Pruebas de control de acceso..... 47
23.1 Usuario de solo lectura..... 47
23.2 Usuario de la aplicación..... 47
23.3 Sin credenciales y sin cifrado..... 48
23.4 Verificación automática en la aplicación..... 48
24. Limitaciones y trabajo futuro..... 48
25. Conclusión..... 49
Referencias..... 50
Anexo. Repositorio y archivos principales..... 51

1. Introducción

Melodic es una plataforma de música en *streaming* construida con Laravel 12 (Laravel, s.f.). En ella, oyentes con suscripción escuchan canciones y las organizan en *playlists*; artistas y sellos discográficos publican música y cobran regalías por cada reproducción, y un administrador supervisa las finanzas de la plataforma.

Este manual documenta la migración de la capa de persistencia de Melodic desde un modelo relacional (MySQL, con SQLite en desarrollo) hacia MongoDB, junto con el endurecimiento de seguridad del sistema y su publicación en línea. También funciona como guía de uso: explica cómo instalar el entorno, cómo operar la base de datos y cómo recorrer la plataforma con cada tipo de cuenta.

El trabajo se planeó como un *sprint* Scrum de dos semanas con dos integrantes y 28 tareas agrupadas en seis épicas: migración de datos, control de acceso, cifrado en tránsito, *hardening* de red, consultas CRUD y comandos de administración. Las tareas se entregaron en 11 *pull requests*; después, el sistema se publicó en Render con MongoDB Atlas y se corrigieron los errores encontrados en producción.

1.1 Objetivo general

Lograr que Melodic funcione sobre MongoDB con los datos migrados y verificados, conexiones cifradas, una red endurecida y la posibilidad de volver a la base relacional si el corte falla.

1.2 Objetivos específicos

- Analizar el modelo relacional y las consultas reales de la aplicación para diseñar las colecciones a partir de sus patrones de acceso.

- Implementar un proceso ETL repetible y verificable que copie los datos de MySQL a MongoDB sin duplicarlos.
- Corregir los defectos del sistema origen relacionados con el dinero (condiciones de carrera, montos en `float` y retiros duplicados).
- Configurar usuarios de mínimo privilegio, TLS en todas las conexiones, una red interna para MongoDB, un *firewall* y acceso administrativo por VPN.
- Publicar la aplicación en Render con MongoDB Atlas y documentar su operación.

1.3 Alcance de este documento

El manual integra la documentación técnica del repositorio (inventario, diseño, corte, consultas, *hardening*, comandos y despliegue) y el manual de usuario. Cubre tanto el entorno local con Docker Compose como el despliegue en la nube. El código fuente está disponible en el repositorio del proyecto (Morales Villarauz y Soberano Paredes, 2026a) y la aplicación publicada puede consultarse en línea (Morales Villarauz y Soberano Paredes, 2026b).

2. Sistema origen: Melodic

2.1 Descripción del negocio

Melodic se comporta como un servicio de *streaming* musical con cuatro tipos de cuenta. Los oyentes pagan una suscripción, escuchan música, buscan canciones y arman sus *playlists*. Los artistas suben álbumes y canciones. Los sellos discográficos (*labels*) administran a varios artistas y cobran por ellos. El administrador consulta cuánto dinero entra y sale de la plataforma y da de alta a los sellos.

Cada vez que alguien escucha una canción, el artista (o su sello, si pertenece a uno) gana 0.15 en regalías. Cuando el artista o el sello retira su saldo, recibe el 70 % y la plataforma conserva el 30 % como comisión.

2.2 Problemática que resuelve el sistema

La plataforma necesita registrar con exactitud tres flujos de dinero: el pago de suscripciones, la acumulación de regalías por reproducción y el retiro de saldos. En el sistema original, la reproducción y el retiro se implementaban leyendo, sumando y guardando el saldo, de modo que dos operaciones simultáneas podían perder incrementos o cobrar dos veces el mismo dinero. La migración se aprovechó para que estas operaciones fueran atómicas.

2.3 Arquitectura actual

Tabla 1

Componentes del sistema antes de la migración

Elemento	Antes de la migración
Aplicación	Laravel 12 (PHP 8.2+), vistas Blade, Tailwind y Chart.js
Base de datos	MySQL 8.4 en Docker (fuente oficial); SQLite en desarrollo y pruebas
Sesiones, caché y colas	Driver `database` sobre la misma base relacional
Roles	`user` (oyente), `artist`, `label` y `admin`, con los *middleware* `role`, `subscribed` y `admin`
Despliegue	Docker Compose: `app` (puerto 8000), `mysql` (3306) y un `mongo:8.0` reservado sin uso
Pruebas	Solo los ejemplos de Laravel

2.4 Roles y módulos

Tabla 2

Tipos de cuenta de Melodic

Cuenta	Qué puede hacer	Cómo se crea
Oyente	Suscribirse, escuchar, explorar y crear *playlists*	Se registra solo en la página
Artista independiente	Subir álbumes y canciones, ver su saldo y retirarlo	Se registra solo en la página
Sello (*label*)	Crear artistas de su sello, ver las reproducciones de sus artistas y retirar el saldo	Lo crea el administrador
Administrador	Ver ingresos, egresos y la gráfica mensual; crear sellos	Viene creado de inicio

Nota. Un artista que pertenece a un sello no cobra directamente: sus regalías se acumulan en el saldo del sello.

Los flujos principales son el registro e inicio de sesión, la suscripción (que genera un ingreso), la reproducción (que suma 0.15 a la regalía del artista o de su sello), el retiro (70 % al artista o sello y 30 % de comisión), la gestión de *playlists*, la búsqueda y los paneles por rol.

3. Justificación de la migración a NoSQL

3.1 Por qué MongoDB es adecuado para Melodic

La decisión se tomó a partir de cómo lee y escribe la aplicación, no como una conversión mecánica de tablas a colecciones. MongoDB guarda la información como documentos BSON, permite embeber datos que se leen juntos y ofrece operadores de actualización atómica sobre un solo documento, además de transacciones multidocumento en un *replica set* (MongoDB, Inc., s.f.-a).

- **Reproducir es la operación más frecuente.** En SQL se hacía leyendo, sumando y guardando, con lo que se perdían incrementos concurrentes. En MongoDB es un `$inc` atómico dentro de una transacción.
- **Las playlists se leen completas.** El pivote `playlist_song` se convierte en un arreglo dentro del documento: mostrar, contar y añadir una canción se resuelve en un solo documento, y añadir sin duplicados es una única escritura condicionada.
- **La búsqueda filtra por nombre de canción y de artista.** Con una copia del artista dentro de cada canción, la búsqueda no necesita unir tablas.
- **El retiro necesita leer y vaciar el saldo en un solo paso.** `findOneAndUpdate` lo hace de forma atómica.

- **El modelo es flexible** para crecer (por ejemplo, con estadísticas por reproducción) sin migraciones de esquema rígidas.

3.2 Qué se gana y qué se cede

Tabla 3

Ventajas y concesiones de la migración

Se gana	Se cede
Reproducciones y retiros atómicos, sin condiciones de carrera	Las referencias entre colecciones no tienen integridad referencial en la base; la valida la aplicación
Lectura de una *playlist* completa en un documento	La copia del artista en cada canción debe propagarse si algún día cambia su nombre
Búsqueda sin uniones entre tablas	La búsqueda por texto usa expresiones regulares y recorre la colección
Dinero en `Decimal128`, sin errores de redondeo	Se necesita un *replica set* (aunque sea de un nodo) para usar transacciones
Un solo servicio de monedero (`WalletService`)	Sesiones, caché y colas siguen en una base relacional

La migración también corrigió problemas del sistema origen: condiciones de carrera en el dinero, tres implementaciones distintas del retiro, montos calculados con `float` y un panel de administración que no funcionaba en SQLite.

4. Análisis del modelo relacional actual

4.1 Inventario de tablas

El inventario se obtuvo de `database/migrations`, `app/Models` y `app/Http/Controllers` en la rama `dev`. La base de origen para la carga es MySQL; localmente se usa SQLite con el mismo esquema.

Tabla 4

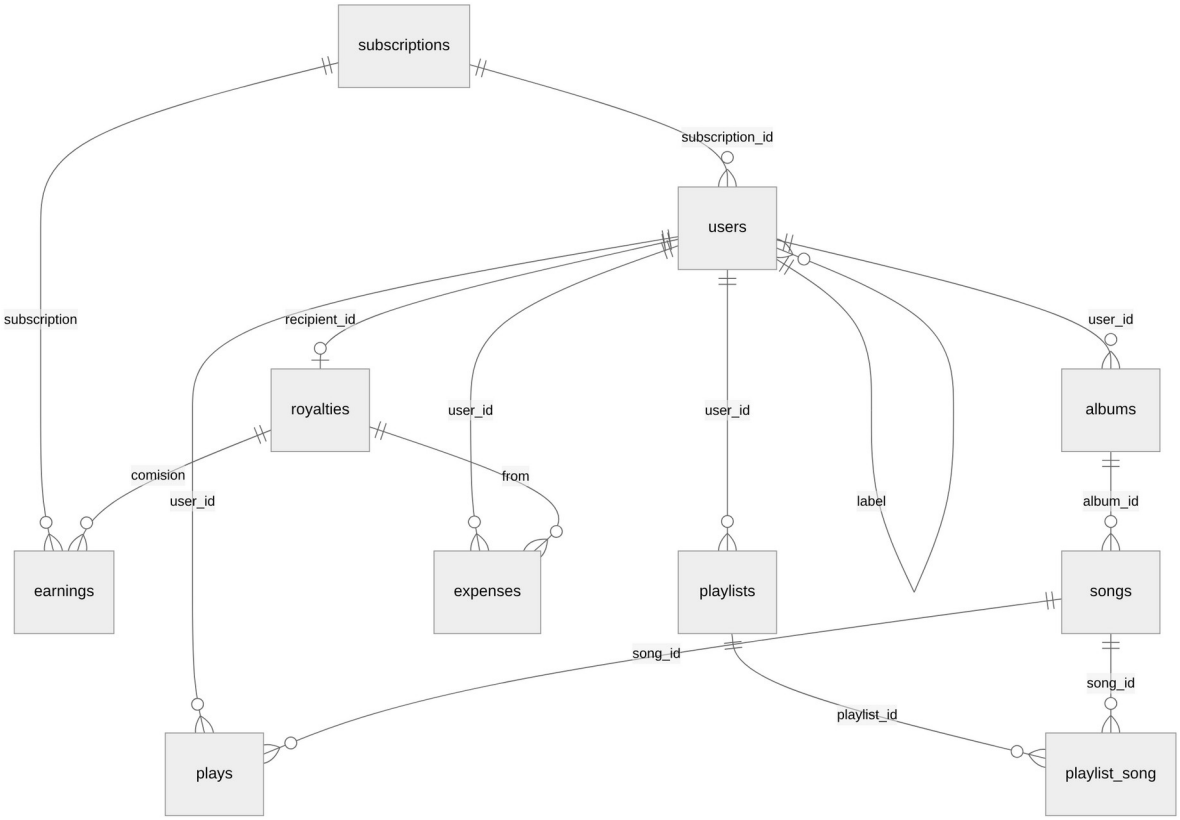
Tablas de dominio del modelo relacional

Tabla	PK	Columnas	Relaciones	Notas del código
`subscriptions`	`id` int	`name`, `cost` decimal(10,2), `months` tinyint	1:N con `users` y `earnings`	Catálogo; lo llena `SubscriptionSeeder`
`users`	`user_id` UUID	`username`, `email` único, `password`, `role`, `is_suscribed`, `fin_subscription`, `path`	`subscription_id` → `subscriptions`; `label` → `users`	El UUID lo genera el modelo
`albums`	`album_id` int	`title`, `path`	`user_id` → `users`	Sin *timestamps*
`songs`	`song_id` int	`name`, `length` (siempre 0), `plays`, `file_path`, `image_path`	`album_id` → `albums`	`length` no se guarda
`playlists`	`playlist_id` int	`name`, `description`, `path`	`user_id` → `users`	`description` nunca se escribe
`playlist_song`	sin PK	*timestamps*	N:M entre `playlists` y `songs`	`attach()` no llena los *timestamps*
`plays`	`play_id` int	*timestamps*	`song_id`, `user_id`	Ningún código la escribe
`royalties`	`royalties_id` int	`amount` decimal(13,2), `last_payment`	`recipient_id` → `users`, único	Un monedero por artista o sello
`earnings`	`earning_id` int	`amount` decimal(13,2), `state`, `at`	`subscription`, `comision`	Solo inserción
`expenses`	`expense_id` int	`amount` decimal(13,2), `state`, `at`	`from` → `royalties`; `user_id` → `users`	Solo inserción

Nota. Las tablas del *framework* (`sessions`, `cache`, `cache_locks`, `jobs`, `job_batches`, `failed_jobs`, `password_reset_tokens` y `migrations`) no se migraron.

Figura 1

Diagrama entidad-relación del modelo relacional de Melodic



Nota. Elaborado a partir del inventario de `docs/mongodb/01-inventario.md`.

4.2 Diferencias entre la documentación y el repositorio

Al revisar el plan del *sprint* contra el código real se encontraron hallazgos que afectaban la migración. La Tabla 5 resume su estado al cierre.

Tabla 5

Hallazgos de la revisión del código

#	Hallazgo	Estado
1	El registro asignaba el rol desde el formulario	Resuelto en E2-T15
2	Secretos en `compose.yaml` y MongoDB publicado	Resuelto en E2-T18 y E4-T21
3	Condiciones de carrera al reproducir y retirar	Resuelto en E1-T11 con `$inc` y `findOneAndUpdate` en transacción

#	Hallazgo	Estado
4	Tres implementaciones del retiro	Resuelto en E1-T11: un solo `WalletService`
5	`DATE_FORMAT` rompía el panel de administración en SQLite	Resuelto en E1-T12 con `$dateToString`
6	Montos calculados con `float`	Resuelto en E1-T11 con bcmath y `Decimal128`
7	La tabla `plays` nunca se escribe	Vigente; *backlog* C01
8	El `down()` de `create_songs_table` borra `songs` antes que `plays`	Vigente; *backlog* C04
9	`SongController@store` sin transacción	Atendido en E1-T09
10	No había pruebas	Resuelto: 42 pruebas funcionales
11	`albums` y `playlist_song` no guardan *timestamps*	La carga acepta `created_at` nulo

4.3 Patrones de acceso

Las frecuencias se estimaron así: muy alta (cada reproducción), alta (cada visita de un oyente), media (paneles de artista y sello) y baja (operaciones puntuales).

Tabla 6

Consultas principales por controlador

Controlador	Ruta	Operación	Frecuencia
`SongController@registerPlay`	`POST /song/{song}/play`	`plays + 1` y regalía + 0.15 en transacción	Muy alta
`DashboardController@user`	`GET /dashboard/user`	6 canciones al azar y *playlists* con conteo	Alta
`DashboardController@explore`	`GET /explore`	Búsqueda por canción o artista; sin término, por fecha	Alta
`PlaylistController@show`	`GET /playlist/{playlist}`	Canciones de la *playlist* y búsqueda de las que faltan	Alta
Middleware `subscribed`	Rutas de oyente	Lee y, si venció, limpia la suscripción	Alta
`PlaylistController@addSong`	`POST /playlist/{playlist}/add-song`	Comprueba si ya existe y la añade	Media
`DashboardController@artist`	`GET /dashboard/artist`	Álbumes con conteo, saldo y últimos 5 retiros	Media
`DashboardController@label`	`GET /dashboard/label`	Artistas del sello, suma de reproducciones y saldo	Media

Controlador	Ruta	Operación	Frecuencia
`LoginController@login`	`POST /login`	Busca por `email`	Media
`WalletController@withdraw`	`POST /withdraw`	Lee el saldo, registra 70 % y 30 % y lo vacía	Baja
`SubscriptionController@store`	`POST /premium`	Inserta un ingreso y actualiza al usuario	Baja
`AdminController@index`	`GET /dashboard/admin`	Totales, ingresos por mes y últimos 10 sellos	Baja

5. Estrategia de modelado

5.1 Criterios

Las colecciones se diseñaron a partir de las consultas de la Tabla 6, siguiendo las recomendaciones de MongoDB sobre cuándo embeber y cuándo referenciar (MongoDB, Inc., s.f.-b):

- **Embeber** cuando los datos se leen juntos y crecen de forma acotada: las canciones de una *playlist* (máximo 500) y la copia del artista en cada canción.
- **Referenciar** cuando la entidad se consulta sola o cambia por su cuenta: álbumes, usuarios, planes, regalías y movimientos contables.
- **Colección propia para el saldo** (`royalties`): cada reproducción hace `$inc` sobre él y, embebido en `users`, competiría con el inicio de sesión y la suscripción.

5.2 Decisión principal: playlists con canciones embebidas

La tabla pivote `playlist_song` desaparece y se convierte en el arreglo `songs[]` dentro de cada *playlist*. El arreglo guarda solo la referencia `song_id` y la fecha `added_at`, no la canción completa. Mostrar una *playlist*, contar sus canciones y añadir una nueva se resuelve sobre un único documento, y la inserción sin duplicados es un `updateOne` atómico.

Para que el documento no crezca sin control, cada *playlist* admite como máximo 500 canciones. El límite se valida en la misma escritura, sin leer antes:

```
db.playlists.updateOne(
  { _id: playlistId, user_id: userId,
    "songs.song_id": { $ne: songId }, "songs.499": { $exists: false } },
  { $push: { songs: { song_id: songId, added_at: new Date() } } }
)
```

Un resultado `matchedCount: 0` indica que la canción ya estaba, que la *playlist* está llena o que no pertenece al usuario.

5.3 Patrones aplicados

Tabla 7

Decisiones de embebido o referencia

Relación	Decisión	Justificación
`playlist_song` → `playlists.songs[]`	Embebido (referencias)	Mostrar, contar y añadir se resuelven en un documento
`artist` → `songs`	Copia embebida (*snapshot*)	Explorar y buscar filtran por `artist.username` sin `$lookup`; reproducir obtiene el destinatario sin leer el álbum
`songs.album_id` → `albums`	Referencia	Las vistas usan la relación `album()`; un campo `album` la ocultaría
`albums.user_id` → `users`	Referencia	Un artista tiene pocos álbumes y se consultan solos
`users.label` → `users`	Referencia	Embeber artistas en el sello duplicaría usuarios
`users.subscription_id` → `subscriptions`	Referencia	Catálogo pequeño que cambia aparte
`royalties.recipient_id` → `users`	Referencia, colección propia	El `$inc` frecuente no reescribe al usuario
`earnings` y `expenses`	Referencia	Libros contables de solo inserción, agregados por fecha y usuario

La copia del artista en `songs` (`artist: { user_id, username }`) la escribe el modelo `Song` cada vez que cambia `album_id`. Hoy no hay flujos que editen `users.username`; si se agregan, deben actualizar la copia en la misma operación con `db.songs.updateMany({ "artist.user_id": id }, { $set: { "artist.username": ... } })`.

5.4 Reglas de negocio actualizadas

- Cada reproducción incrementa `songs.plays` y suma 0.15 al monedero del artista o de su sello dentro de una sola transacción: si falla la regalía, no se cuenta la reproducción.
- El retiro lee y vacía el saldo en un paso con `findOneAndUpdate` y, con el saldo anterior, registra un egreso del 70 % y un ingreso del 30 %.
- El dinero se calcula con bcmath y se guarda como `Decimal128`; el incremento usa `new Decimal128('0.15')` en lugar de un `float`.
- El registro público solo crea cuentas de oyente o de artista; los sellos los crea el administrador.
- Una canción no puede repetirse en la misma *playlist* y cada *playlist* admite hasta 500 canciones.

6. Mapeo SQL → MongoDB

6.1 Mapeo de tablas a colecciones

Tabla 8

Colecciones de la base `melodic`

Colección	Origen	Contenido
`users`	`users`	Cuentas de los cuatro roles
`subscriptions`	`subscriptions`	Catálogo de planes
`albums`	`albums`	Álbumes de cada artista
`songs`	`songs`	Canciones con la copia de su artista
`playlists`	`playlists` + `playlist_song`	*Playlists* con sus canciones embebidas
`royalties`	`royalties`	Un monedero por destinatario
`earnings`	`earnings`	Ingresos de la plataforma, solo inserción
`expenses`	`expenses`	Pagos a artistas y sellos, solo inserción

Nota. La tabla `plays` no se migra: ningún código la escribe y el contador vive en `songs.plays`.

6.2 Mapeo campo por campo

`users._id` conserva el UUID original como texto. Las demás colecciones usan `ObjectId` y guardan `legacy_id` (el entero original) durante la transición, para resolver las llaves foráneas en la carga y comparar contra la base relacional. Las referencias a otros documentos se guardan como el texto hexadecimal del `ObjectId`, porque es la convención de laravel-mongodb: con un `ObjectId` las relaciones `hasMany` no encontraban registros (MongoDB, Inc., s.f.-c).

Tabla 9

Conversión de tipos

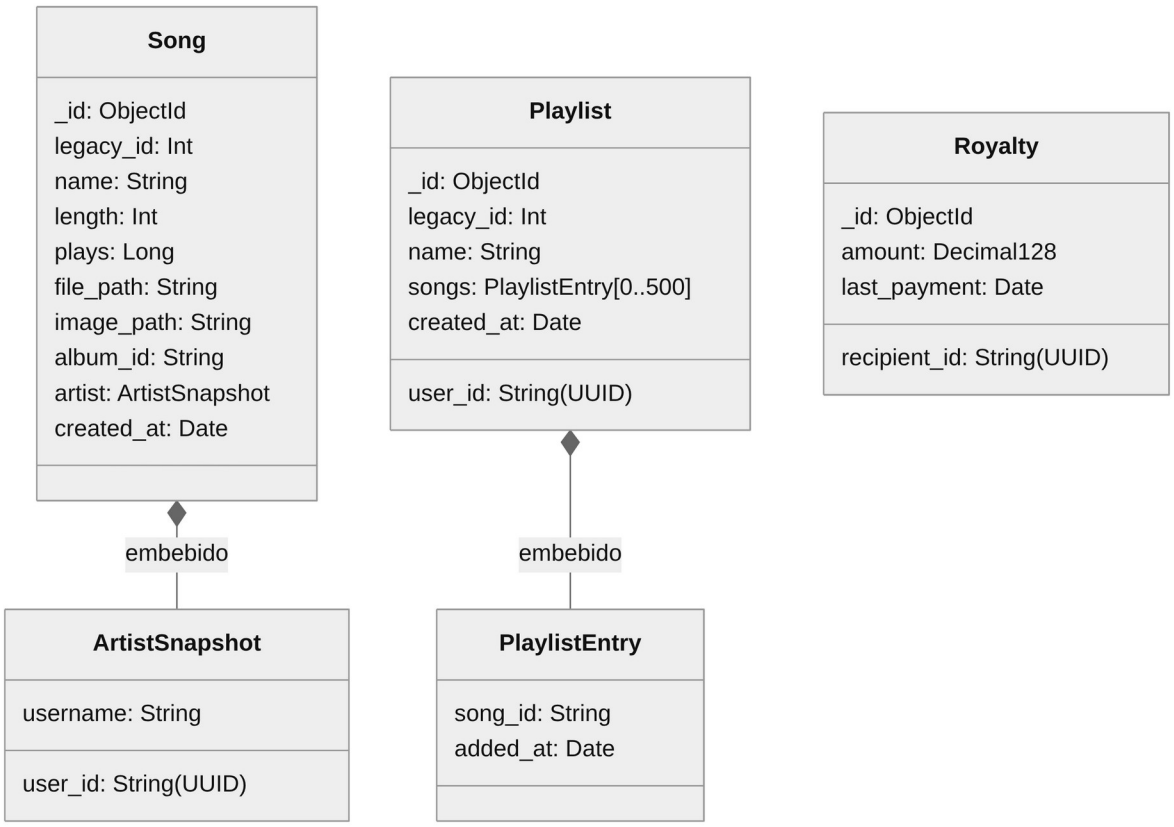
Origen (MySQL / SQLite)	**MongoDB**	**Regla**
PK int (`id`, `album_id`, `song_id`…)	`ObjectId` en `_id` e `Int32` en `legacy_id`	Se genera un `ObjectId` nuevo; el entero se guarda aparte
UUID (`user_id`, `label`, `recipient_id`)	String	Sin cambios
FK int (`album_id`, `subscription_id`, `from`, `comision`…)	String con el hexadecimal del `ObjectId`	Se busca el documento por `legacy_id`
`decimal(10,2)` y `decimal(13,2)`	`Decimal128`	Desde el texto de la base, sin pasar por `float`
`timestamp`	`Date` UTC	La aplicación usa UTC
`date` (`fin_subscription`)	`Date` a las 00:00 UTC	—
`boolean` (`is_suscribed`)	`Boolean`	—
`unsignedBigInteger` (`plays`)	`Int64`	—
`tinyint` / `int`	`Int32`	—
`NULL`	`null` explícito	Conserva `whereNull` y `=== null`

Se descartó guardar el dinero en centavos, porque obligaba a cambiar cada vista y cada cálculo. Con el *cast* `decimal:2` de laravel-mongodb, el modelo devuelve un texto (por ejemplo, `'10.30'`) que funciona con `number_format` en Blade.

7. Diagrama de documentos

Figura 2

Estructura de los documentos con datos embebidos



Nota. Las composiciones marcan los subdocumentos embebidos: la copia del artista en cada canción y las entradas de cada *playlist*.

7.1 Ejemplos por colección

Los siguientes documentos ilustran la forma final de cada colección.

Colección users

```
{ _id: "9b1c7e2a-4f0d-4c1e-9a55-2f6d3c8b1a01", username: "Carla Ríos",
  email: "carla@melodic.com", password: "$2y$12$...", role: "artist",
  is_suscribed: true, fin_subscription: null, subscription_id: null,
  label: "5e0a9c11-7b3d-4b8e-8f21-0c4d6a9e7f10", path: null,
  created_at: ISODate("2026-10-12T15:04:00Z") }
```

Colección subscriptions

```
{ _id: ObjectId("6710a000000000000000001"), legacy_id: 1, name: "Mensual",
  cost: NumberDecimal("99.00"), months: 1,
  created_at: ISODate("2026-10-12T15:00:00Z") }
```

Colección albums

```
{ _id: ObjectId("6710a000000000000000101"), legacy_id: 7,
  title: "Noches de octubre", path: "album_covers/noches.jpg",
  user_id: "9b1c7e2a-4f0d-4c1e-9a55-2f6d3c8b1a01",
  created_at: null, updated_at: null }
```

Colección songs

```
{ _id: ObjectId("6710a000000000000000201"), legacy_id: 42, name: "Luna nueva",
  length: 0, plays: NumberLong(1280), file_path: "songs/audio/luna.mp3",
  image_path: null, album_id: "6710a000000000000000101",
  artist: { user_id: "9b1c7e2a-4f0d-4c1e-9a55-2f6d3c8b1a01",
            username: "Carla Ríos" },
  created_at: ISODate("2026-10-13T18:20:00Z") }
```

Colección playlists

```
{ _id: ObjectId("6710a000000000000000301"), legacy_id: 3, name: "Para correr",
  description: null, path: null, user_id: "1f4e8d22-0a6b-4c3f-b7e9-5d2a1c0e9b33",
  songs: [ { song_id: "6710a000000000000000201", added_at: null } ],
  created_at: ISODate("2026-10-14T20:00:00Z") }
```

Colección royalties

```
{ _id: ObjectId("6710a000000000000000401"), legacy_id: 2,
  recipient_id: "5e0a9c11-7b3d-4b8e-8f21-0c4d6a9e7f10",
  amount: NumberDecimal("192.00"), last_payment: ISODate("2026-10-16T12:00:00Z") }
```

Colección earnings

```
{ _id: ObjectId("6710a000000000000000501"), legacy_id: 15,
  amount: NumberDecimal("57.60"), state: "pagado",
  at: ISODate("2026-10-16T12:00:00Z"), subscription: null,
  comision: "6710a000000000000000401" }
```

Colección expenses

```
{ _id: ObjectId("6710a000000000000000601"), legacy_id: 9,
  amount: NumberDecimal("134.40"), state: "pagado",
  at: ISODate("2026-10-16T12:00:00Z"), from: "6710a000000000000000401",
  user_id: "5e0a9c11-7b3d-4b8e-8f21-0c4d6a9e7f10" }
```

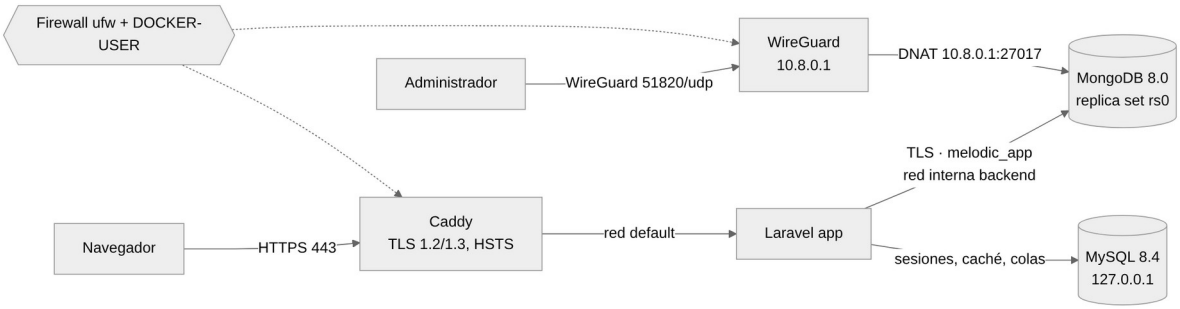
8. Arquitectura de MongoDB

8.1 Componentes

La propuesta inicial fue un *replica set* de un nodo (`rs0`) para disponer de transacciones, con autenticación por *keyFile*, TLS y usuarios de mínimo privilegio; MongoDB quedaría aislado en la red de Docker y accesible para administración solo por VPN. La Figura 3 muestra la arquitectura final del entorno local.

Figura 3

Arquitectura final del entorno local con Docker Compose



Nota. Caddy termina HTTPS; MongoDB vive en una red interna sin puertos publicados y solo se alcanza por la VPN para administración.

8.2 Opciones de despliegue evaluadas

Tabla 10

Diferencias entre la arquitectura propuesta y la final

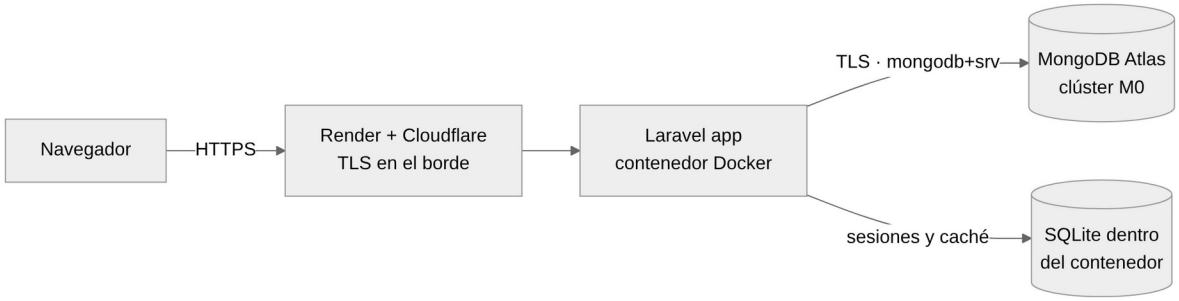
Aspecto	Propuesta	Final y motivo
Topología	*Replica set* de un nodo	Igual, con *keyFile*, `requireTLS` y sin puertos publicados
Referencias	`ObjectId`	Texto con el hexadecimal del `ObjectId`, por la convención de laravel-mongodb
Copia del álbum en `songs`	`album{title, path}` y `artist`	Solo `artist`: un campo `album` ocultaba la relación `album()`
Corte	Cambiar `DB_CONNECTION`	Los modelos usan `$connection = 'mongodb'`; `DB_CONNECTION` queda relacional

Aspecto	Propuesta	Final y motivo
Índice de `legacy_id`	Único	Único y *sparse*, porque los documentos nuevos no lo tienen
Migración de índices	`database/migrations`	`database/migrations/mongodb`, para que las pruebas no la ejecuten
Pruebas	SQLite en memoria	Base `melodic_testing` en MongoDB con su propio usuario
VPN	WireGuard en el *host*	WireGuard en un contenedor opcional (`compose.vpn.yaml`)
Secretos	Secretos de Docker Compose	Volumen `melodic-nosql_mongodb-secrets`, para que funcione con SELinux
Producción	No prevista	Render (Docker) con MongoDB Atlas

En producción, MongoDB es un clúster de Atlas (*replica set* administrado, con TLS de fábrica) y las sesiones y la caché usan SQLite dentro del contenedor, porque el plan gratuito de Render no incluye una base relacional ni un disco persistente (Render, s.f.).

Figura 4

Arquitectura de producción en Render y MongoDB Atlas



Nota. El TLS del navegador termina en el borde de Render y Cloudflare.

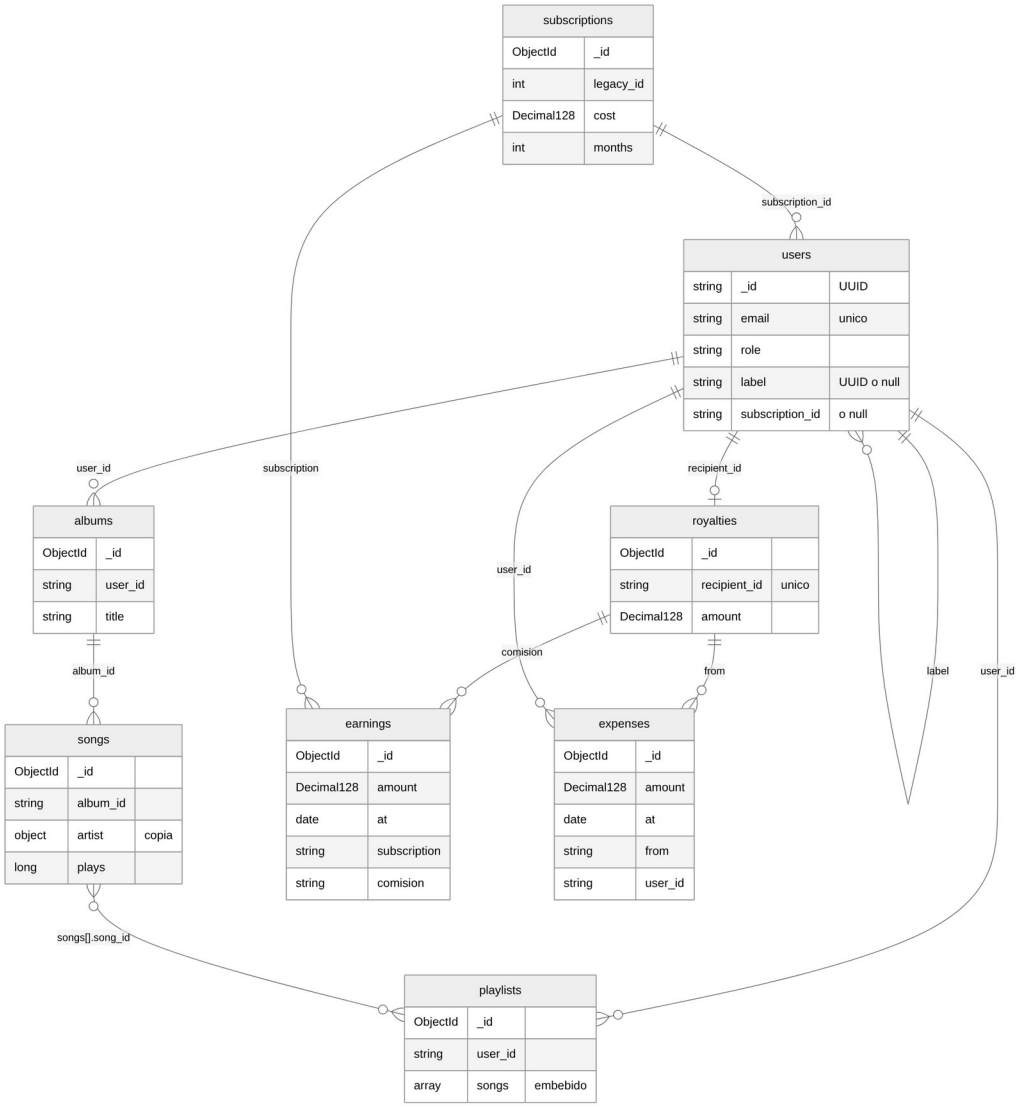
9. Diagrama del modelo de datos (colecciones)

9.1 Relaciones entre colecciones

La Figura 5 resume las ocho colecciones y las referencias entre ellas. Las líneas representan referencias que valida la aplicación, no llaves foráneas de la base.

Figura 5

Modelo de datos de las colecciones de MongoDB



Nota. La relación entre `songs` y `playlists` se resuelve con el arreglo embebido `songs[].song_id`.

10. Índices y validación de esquema

10.1 Índices

Los 18 índices se crean con una migración versionada (`database/migrations/mongodb/2026_10_14_000000_create_mongodb_collections.php`) y cada uno responde a una consulta real.

Tabla 11

Índices de la base melodic

Colección	Índice	Consulta que lo justifica
`users`	`{ email: 1 }` único	Inicio de sesión y unicidad del correo
`users`	`{ label: 1 }`	Artistas de un sello
`users`	`{ role: 1, created_at: -1 }`	Últimos 10 sellos del panel de administración
`albums`	`{ user_id: 1 }`	Álbumes del artista
`songs`	`{ album_id: 1 }`	Canciones de un álbum y conteo por álbum
`songs`	`{ "artist.user_id": 1 }`	Reproducciones por artista y por sello
`songs`	`{ created_at: -1 }`	Explorar sin búsqueda
`playlists`	`{ user_id: 1, created_at: -1 }`	*Playlists* del usuario por fecha
`royalties`	`{ recipient_id: 1 }` único	Saldo en cada reproducción y retiro; un monedero por destinatario
`earnings`	`{ at: 1 }`	Ingresos por mes y de los últimos 30 días
`expenses`	`{ user_id: 1, at: -1 }`	Últimos 5 retiros
7 colecciones	`{ legacy_id: 1 }` único y *sparse*	Carga idempotente y verificación

Solo tres consultas recorren la colección completa, de forma aceptada: la búsqueda con expresión regular (un `LIKE '%q%'` tampoco usaba índice en SQL), la selección de 6 canciones al azar con `$sample` y los totales de ingresos y la lista de planes, que son de baja frecuencia sobre colecciones pequeñas.

10.2 Validación de esquema

No se configuraron validadores `$jsonSchema` en las colecciones. La validación vive en la aplicación y en los índices:

- Reglas de Laravel en cada formulario, con `Rule::unique` y `Rule::exists` sobre los modelos de MongoDB.
- Índices únicos para el correo, el monedero por destinatario y `legacy_id`.
- Valores por defecto en los modelos (rol, suscripción y nulos explícitos), porque MongoDB no tiene valores por defecto de columna.
- El límite de 500 canciones se valida dentro de la misma escritura con `{ "songs.499": { $exists: false } }`.

Agregar validadores `$jsonSchema` por colección queda como trabajo futuro.

11. Estrategia de migración (ETL)

11.1 Fases

Tabla 12

Fases del proceso ETL

Fase	Implementación
Extracción	`melodic:migrate-to-mongo` lee de la conexión relacional (`--from`, por defecto MySQL) en el orden subscriptions → users → albums → songs → playlists → royalties → earnings → expenses
Transformación	Enteros a `ObjectId` más `legacy_id`; llaves foráneas a la referencia del documento; decimales a `Decimal128` sin pasar por `float`; fechas a UTC; `playlist_song` a arreglo; copia del artista en cada canción
Carga	*Upsert* por `legacy_id` (por UUID en `users`) en lotes de 500. Se puede repetir sin duplicar
Verificación	`melodic:verify-mongo` compara conteos, montos al centavo, reproducciones, canciones por *playlist*, referencias huérfanas y una muestra campo por campo

11.2 Esquema del script

Primero se simula la carga y después se ejecuta. La opción `--collection=songs` carga una sola colección (sus referencias ya deben estar cargadas) y `--from=sqlite` cambia la conexión de origen. El comando termina con código 1 si alguna fila tuvo errores.

```
docker exec melodic-app php artisan migrate --path=database/migrations/mongodb --force
docker exec melodic-app php artisan melodic:migrate-to-mongo --dry-run
docker exec melodic-app php artisan melodic:migrate-to-mongo
```

11.3 Validaciones posteriores a la carga

La verificación termina con código 1 si encuentra diferencias entre MongoDB y la base relacional:

```
docker exec melodic-app php artisan melodic:verify-mongo
```

Para revertir las colecciones e índices y empezar desde cero se usa `migrate:rollback --path=database/migrations/mongodb --force`.

11.4 Corte y reversión

Los modelos de dominio usan `$connection = 'mongodb'`, así que el corte consiste en desplegar ese código con los datos ya cargados. `DB_CONNECTION` se queda en la base relacional, donde siguen la tabla `migrations`, las sesiones, la caché y las colas; cambiarla a `mongodb` rompería `php artisan migrate`.

El corte se hace solo si se cumplen tres criterios (*go / no-go*):

1. `melodic:verify-mongo` termina con 0 diferencias.
2. `composer test` pasa sobre MongoDB.
3. El recorrido con los 4 roles funciona: oyente, artista, sello y administrador.

```
docker exec melodic-app php artisan down
docker exec melodic-mysql sh -c 'mysqldump -umelodic -p"$MYSQL_PASSWORD" --single-
transaction --no-tablespaces melodic' > respaldo-melodic.sql
docker exec melodic-mongodb sh -c 'mongodump --quiet --uri "mongodb://melodic_backup:
$MONGO_BACKUP_PASSWORD@127.0.0.1:27017/?authSource=admin&tls=true&tlsCAFile=/run/
secrets/mongodb_tls_ca" --db melodic --archive --gzip' > respaldo-mongo.archive.gz
git switch <rama con el corte>
docker compose up --build --detach app
docker exec melodic-app php artisan view:clear
docker exec melodic-app php artisan migrate --path=database/migrations/mongodb --force
docker exec melodic-app php artisan melodic:migrate-to-mongo
docker exec melodic-app php artisan melodic:verify-mongo
composer test
docker exec melodic-app php artisan up
```

El paso `view:clear` es obligatorio, porque el volumen `app-storage` conserva las vistas compiladas del despliegue anterior. La reversión consiste en volver a desplegar el código anterior; como la carga no modifica la base relacional, la aplicación regresa con los datos que tenía antes del corte. Lo escrito en MongoDB después de reabrir el tráfico requiere conciliación manual.

```
docker exec melodic-app php artisan down
git switch <rama anterior al corte>
docker compose up --build --detach app
docker exec melodic-app php artisan view:clear
docker exec melodic-app php artisan up
```

12. Seguridad y hardening

12.1 Controles implementados

Tabla 13

Controles de seguridad por capa

Capa	Medida	Tarea
Aplicación	El registro público solo crea oyentes y artistas	E2-T15
Aplicación	5 intentos por minuto en inicio de sesión y registro, con contadores separados	E2-T16
Secretos	Contraseñas y `APP_KEY` en `.env`; el `APP_KEY` publicado se rotó	E2-T18
Base de datos	Usuarios `melodic_app`, `melodic_readonly`, `melodic_backup` y `melodic_test` con SCRAM-SHA-256	E2-T17
Cifrado	MongoDB con `requireTLS`, solo TLS 1.2/1.3 y suites con secreto perfecto hacia adelante	E3-T19, E4-T24
Cifrado	HTTPS con Caddy, HSTS y *cookies* `secure`	E3-T20

Capa	Medida	Tarea
Red	MongoDB en una red interna sin puertos publicados; MySQL solo en `127.0.0.1`	E4-T21, E4-T22
Red	*Firewall* ufw con reglas en `DOCKER-USER`	E4-T22
Red	Acceso administrativo a MongoDB solo por WireGuard	E4-T23

12.2 Hallazgos de seguridad en el sistema original

El sistema original tenía secretos escritos en `compose.yaml`, MongoDB publicado en el *host*, el puerto 8000 de la aplicación y el 3306 de MySQL abiertos aunque ufw estaba activo, y un registro que aceptaba cualquier rol desde el formulario. Todos se corrigieron en las tareas de las épicas E2 a E4.

12.3 Seguridad local frente a producción

En producción las medidas cambian, porque la infraestructura la administran Render y Atlas.

Tabla 14

Medidas de seguridad en local y en producción

Aspecto	Local (Docker)	Producción (Render y Atlas)
HTTPS	Caddy con TLS 1.2/1.3 y HSTS	TLS en el borde de Render y Cloudflare; `http://` redirige con 301, sin cabecera HSTS
Conexión a MongoDB	TLS con CA propia y `melodic_app`	TLS de Atlas con el usuario creado en Atlas
Red de MongoDB	Red interna, sin puertos	Lista de IP de Atlas abierta (`0.0.0.0/0`), porque Render no tiene IP fijas en el plan gratuito
Firewall y VPN	ufw, `DOCKER-USER` y WireGuard	No aplican: Render no expone el contenedor y Atlas se administra desde su consola

13. Vistas del sistema (frontend existente)

Las vistas se conservaron sin cambios visibles para el usuario: cada modelo expone *accessors* (`song_id`, `album_id`, `playlist_id` y `user_id`) que devuelven su `_id` como texto, de

modo que las plantillas Blade siguieron funcionando. Un reproductor común (`components/player`) permite escuchar desde cualquier vista.

Tabla 15

Vistas de Melodic

Vista	Ruta	Rol	Contenido
Inicio de sesión y registro	`/login`, `/register`	Público	Acceso y alta de oyentes o artistas
Premium	`/premium`	Oyente	Planes de suscripción
Inicio del oyente	`/dashboard/user`	Oyente	6 canciones al azar y sus *playlists*
Explorar	`/explore`	Suscritos	Catálogo y búsqueda por canción o artista
Mis *playlists* y detalle	`/playlists`, `/playlist/{id}`	Oyente	*Playlists*, sus canciones y buscador para añadir
Álbum	`/album/{id}`	Suscritos	Canciones del álbum
Panel del artista	`/dashboard/artist`	Artista	Álbumes, subida de canciones, saldo y retiros
Panel del sello	`/dashboard/label`	Sello	Artistas, reproducciones totales, saldo y retiros
Panel de administración	`/dashboard/admin`	Admin	Ingresos, egresos, ganancia, proyección, gráfica mensual y alta de sellos

14. Implementación y evidencias

14.1 Qué se logró

La migración se completó en un *sprint* con las 28 tareas cerradas. El trabajo quedó registrado en los *pull requests* del repositorio.

Tabla 16

Pull requests del proyecto

PR	Contenido
#1	*Replica set* y conexión de Laravel (E1-T04)
#2	Secretos fuera de `compose.yaml` (E2-T18)

PR	Contenido
#3	Roles del registro público (E2-T15)
#4	Límite de intentos (E2-T16)
#5	Pruebas funcionales base (E1-T06)
#6	Usuarios de mínimo privilegio (E2-T17)
#7	Red interna para MongoDB (E4-T21)
#8	TLS en MongoDB (E3-T19)
#9	Inventario, diseño, índices, carga y catálogo CRUD (E1-T01 a E1-T07, E5-T25)
#10	Verificación, modelos, *playlists*, dinero, panel, búsqueda y corte (E1-T08 a E1-T14)
#11	HTTPS, *firewall*, VPN, *hardening*, *explain*, respaldo y *runbook* (E3-T20, E4-T22 a E4-T24, E5-T26, E6-T27, E6-T28)
#12	Manual de usuario e informe técnico
#13	Ajustes de Docker Compose para SELinux (Fedora)
#14, #16	Despliegue en Render con MongoDB Atlas (`render.yaml`)
#15, #17, #19	Paso de `dev` a `main` para desplegar
#18	Correcciones de producción: planes, *scroll* del panel y subida de canciones

14.2 Ramas del repositorio

Cada tarea o grupo de tareas se desarrolló en su propia rama y se integró a `dev` por *pull request*; `main` es la rama que despliega Render.

Tabla 17

Ramas del repositorio Melodic-NoSQL

Rama	Propósito
`main`	Rama de producción que despliega Render
`dev`	Rama de integración del *sprint*
`feat/e1-t01-t07`	Inventario, diseño, índices, carga y catálogo CRUD
`feat/e1-t04`	*Replica set* y conexión de Laravel con MongoDB
`feat/e1-t08-t14`	Verificación, modelos, dinero, panel, búsqueda y corte
`test/e1-t06`	Pruebas funcionales base
`fix/e2-t15`, `fix/e2-t16`, `fix/e2-t18`	Roles del registro, límite de intentos y secretos
`feat/e2-t17`	Usuarios de mínimo privilegio
`feat/e3-t19`	TLS en MongoDB

Rama	Propósito
`feat/e3-t20-e6-t27`	HTTPS, *firewall*, VPN, *hardening* y *runbook*
`feat/e4-t21`	Red interna para MongoDB
`fix/docker-fedora-selinux`	Compatibilidad de Docker Compose con SELinux
`feat/deploy-render-atlas`	Despliegue en Render con MongoDB Atlas
`fix/render-production`	Correcciones encontradas en producción
`docs/manuales`	Manual de usuario e informe técnico

Nota. Ramas obtenidas del repositorio público el 20 de octubre de 2026; pueden consultarse en https://github.com/Theproxy206/Melodic-NoSQL/branches

15. Arquitectura final

En el entorno local, todos los servicios corren con Docker Compose en un solo equipo.

Tabla 18

Servicios del entorno local

Servicio	Función	Exposición
`proxy` (Caddy 2.10)	HTTPS y redirección desde `http://`	80 y 443
`app` (Laravel)	Aplicación	Solo detrás del *proxy* (`app:8000`)
`mongodb` (8.0)	Datos de dominio	Red interna (`mongodb:27017`); `127.0.0.1:27018` solo con `compose.localhost.yaml`
`mysql` (8.4)	Sesiones, caché y colas	`127.0.0.1:3306`
`wireguard` (opcional)	VPN administrativa	51820/udp

En producción, la aplicación corre en un Web Service de Render con *runtime* Docker y plan gratuito, configurado con `render.yaml`; los datos viven en un clúster M0 de MongoDB Atlas y las sesiones y la caché en SQLite dentro del contenedor.

16. Datos y migración

16.1 Datos sintéticos

El ensayo de corte se hizo en el entorno Docker local sobre la base MySQL `melodic`, sembrada con los *seeders* y datos de *factories*: 13 usuarios, 18 canciones, 5 *playlists*, 2 regalías, 6 ingresos y 2 egresos, más las escrituras de un recorrido previo.

16.2 Resultado

El primer intento fue *no-go*: el recorrido con los 4 roles devolvió errores 500 en la *playlist* y en el panel del artista. Las causas fueron una vista compilada del código anterior guardada en el volumen `app-storage` y el uso de `withCount('songs')`, que laravel-mongodb no soporta. Ambas se corrigieron y se agregaron pruebas. Después se ensayó la reversión a MySQL y, finalmente, un segundo corte.

Tabla 19

Ensayos de corte del 19 de octubre de 2026

Ensayo	Resultado	Ventana de mantenimiento
Primer corte (01:58)	*No-go*: vistas compiladas antiguas y `withCount`	—
Reversión a MySQL (02:02)	Correcta	31.2 s
Segundo corte (02:04)	*Go*: 62 documentos, 0 diferencias, 42 pruebas y recorrido correcto	50.9 s

Tabla 20

Tiempos del segundo corte

Paso	Tiempo (s)
Modo mantenimiento	0.2
Respaldo de MySQL (`mysqldump`)	0.1
Respaldo de MongoDB (`mongodump`)	0.3

Paso	Tiempo (s)
Desplegar el código nuevo y limpiar vistas	31.5
Colecciones e índices desde cero	0.8
Carga (62 documentos, 0 errores)	0.3
Verificación: 0 diferencias	0.3
`composer test` sobre MongoDB	17.2
Salir de mantenimiento	0.2
Ventana de mantenimiento total	50.9

Nota. El recorrido posterior con los 4 roles tardó 2.0 s y fue correcto.

17. Seguridad: autenticación y control de acceso

17.1 En MongoDB

MongoDB exige autenticación con SCRAM-SHA-256 y cada uso tiene su propio usuario con el permiso mínimo necesario (MongoDB, Inc., s.f.-d).

Tabla 21

Usuarios de MongoDB

Usuario	Permisos	Uso
`melodic_app`	`readWrite` sobre `melodic`	La aplicación
`melodic_readonly`	`read` sobre `melodic`	Consultas y reportes
`melodic_backup`	`backup`	`mongodump`
`melodic_test`	`readWrite` sobre `melodic_testing`	`composer test`; solo se crea si `MONGO_TEST_PASSWORD` tiene valor

Los usuarios se crean solos cuando el volumen está vacío. En un volumen existente, o después de cambiar sus contraseñas en `.env`, se aplican con:

```
docker exec melodic-mongodb sh -c 'mongosh --tls --tlsCAFile
/run/secrets/mongodb_tls_ca --quiet -u "$MONGO_INITDB_ROOT_USERNAME" -p
"$MONGO_INITDB_ROOT_PASSWORD" /docker-entrypoint-initdb.d/01-users.js'
```

Para crear a mano un usuario de solo lectura y borrarlo después:

```
docker exec -it melodic-mongodb sh -c 'mongosh --tls --tlsCAFile
/run/secrets/mongodb_tls_ca --quiet -u "$MONGO_INITDB_ROOT_USERNAME" -p
"$MONGO_INITDB_ROOT_PASSWORD" --authenticationDatabase admin admin --eval
"db.createUser({ user: \"reportes\", pwd: passwordPrompt(), roles: [{ role: \"read\",
db: \"melodic\" }], mechanisms: [\"SCRAM-SHA-256\"] })"'
docker exec melodic-mongodb sh -c 'mongosh --tls --tlsCAFile
/run/secrets/mongodb_tls_ca --quiet -u "$MONGO_INITDB_ROOT_USERNAME" -p
"$MONGO_INITDB_ROOT_PASSWORD" --authenticationDatabase admin admin --eval
"db.dropUser(\"reportes\")"'
```

17.2 En la aplicación

- El registro público solo crea cuentas `user` y `artist`; rechaza `admin` y `label`.
- El inicio de sesión y el registro permiten 5 intentos por minuto, con contadores separados.
- Los *middleware* `role`, `subscribed` y `admin` protegen las rutas por rol; un oyente sin suscripción es enviado a la página Premium.
- Solo el dueño puede ver o modificar su *playlist*.

17.3 En los contenedores

Las contraseñas no se versionan: cada integrante las genera en su archivo `.env`. El *keyFile* del *replica set*, la CA y el certificado TLS se copian con `sync-secrets.sh` al volumen `melodic-nosql_mongodb-secrets`, que `compose.yaml` monta en `/run/secrets`. Se usa un volumen en lugar de secretos de Compose para que funcione con SELinux.

```
openssl rand -hex 24
./docker/mongo/generate-keyfile.sh
./docker/mongo/tls/generate-certs.sh
./docker/mongo/sync-secrets.sh
```

Los volúmenes guardan la contraseña con la que se crearon. Para rotarlas sin borrar datos, con los contenedores corriendo y las contraseñas anteriores todavía en `.env`:

```
docker exec melodic-mongodb sh -c 'mongosh --tls --tlsCAFile
/run/secrets/mongodb_tls_ca --quiet -u "$MONGO_INITDB_ROOT_USERNAME" -p
"$MONGO_INITDB_ROOT_PASSWORD" admin --eval
"db.changeUserPassword(\"$MONGO_INITDB_ROOT_USERNAME\", \"NUEVA\")"'
docker exec melodic-mysql sh -c 'mysql -uroot -p"$MYSQL_ROOT_PASSWORD" -e "ALTER USER
root@\"%\" IDENTIFIED BY \"NUEVA_ROOT\"; ALTER USER root@localhost IDENTIFIED BY
\"NUEVA_ROOT\"; ALTER USER melodic@\"%\" IDENTIFIED BY \"NUEVA_APP\";"'
docker compose up --detach --force-recreate
```

18. Cifrado en tránsito

18.1 Del navegador al servidor

Caddy sirve la aplicación en `https://localhost` con TLS 1.2 y 1.3, HSTS y redirección desde `http://` (Caddy, s.f.). El certificado lo firma su CA interna; para que el navegador y `curl` confíen en él, se exporta la CA y se importa en el navegador o en el sistema (en Arch Linux, con `sudo trust anchor caddy-root.crt`).

```
docker cp melodic-proxy:/data/caddy/pki/authorities/local/root.crt caddy-root.crt
curl -I --cacert caddy-root.crt https://localhost
```

El escaneo con `nmap` del puerto 443 del *proxy* confirmó que solo ofrece suites con calificación A.

Tabla 22

Suites TLS ofrecidas por el proxy Caddy

Versión	Suites
TLS 1.2	`ECDHE_ECDSA` con `AES_128_GCM`, `AES_256_GCM` y `CHACHA20_POLY1305`
TLS 1.3	`AES_128_GCM`, `AES_256_GCM` y `CHACHA20_POLY1305` sobre x25519

Nota. Escaneo realizado con `instrumentisto/nmap:7.95` el 19 de octubre de 2026; menor calificación obtenida: A.

18.2 De la aplicación a MongoDB

MongoDB solo acepta conexiones TLS (`requireTLS`) y deshabilita TLS 1.0 y 1.1 (`--tlsDisabledProtocols TLS1_0,TLS1_1`). En TLS 1.2 se limitaron las suites a intercambio de claves con secreto perfecto hacia adelante y cifrado AEAD (`--tlsCipherConfig ECDHE+AESGCM:ECDHE+CHACHA20:DHE+AESGCM:DHE+CHACHA20`): antes ofrecía 47 suites, 30 de ellas sin secreto perfecto hacia adelante o en modo CBC; después, solo 6.

Tabla 23

Prueba de versiones TLS contra MongoDB

Prueba	Resultado
`openssl s_client -tls1`	Rechazada: `Cipher is (NONE)`
`openssl s_client -tls1_1`	Rechazada: `no protocols available`
`openssl s_client -tls1_2`	Aceptada con `ECDHE-RSA-AES256-GCM-SHA384`
`openssl s_client -tls1_3`	Aceptada con `TLS_AES_256_GCM_SHA384`

La cadena de conexión de la aplicación en desarrollo usa TLS y la CA propia:

```
MONGODB_URI="mongodb://melodic_app:MONGO_APP_PASSWORD@127.0.0.1:27018/melodic?
authSource=admin&directConnection=true&tls=true&tlsCAFile=docker/mongo/tls/ca.pem"
```

Para renovar los certificados se vuelven a correr `generate-certs.sh` y `sync-secrets.sh` y se recrean los contenedores. En producción, Atlas proporciona TLS de fábrica y no requiere `tlsCAFile`.

19. Seguridad de red

19.1 Firewall

El script `docker/firewall/ufw-setup.sh` deja el *host* con entrada denegada salvo 80/tcp, 443/tcp, 51820/udp y el puerto 22 solo desde la red de la VPN (`VPN_SUBNET`, por defecto `172.30.80.0/24`). También agrega a `/etc/ufw/after.rules` las reglas de la cadena `DOCKER-USER`, porque Docker se salta ufw en los puertos publicados (Docker, Inc., s.f.). No borra reglas existentes y se puede repetir; se aplica con una consola local abierta:

```
sudo sh docker/firewall/ufw-setup.sh
sudo ufw status verbose
sudo iptables -S DOCKER-USER
```

Para revertirlo se borra el bloque `MELODIC DOCKER-USER` de `/etc/ufw/after.rules`, se quitan las reglas con `sudo ufw delete allow 80/tcp` (y las demás) y se ejecuta `sudo ufw reload`.

19.2 Puertos expuestos

Tabla 24

Puertos del host antes y después del hardening

Puerto	Antes	Después
22/tcp	filtrado	filtrado
80/tcp	filtrado	abierto
443/tcp	filtrado	abierto
3306/tcp (MySQL)	abierto	filtrado
8000/tcp (app)	abierto	filtrado
27017/tcp (MongoDB)	filtrado	filtrado
27018/tcp (MongoDB local)	filtrado	filtrado
51820/udp (WireGuard)	abierto o filtrado	abierto o filtrado

Nota. Escaneo desde un contenedor hacia la IP del *host* (192.168.1.66). En el estado anterior, 3306 y 8000 estaban abiertos aunque ufw estaba activo, porque Docker publica los puertos con sus propias reglas. WireGuard aparece como abierto o filtrado porque no responde a paquetes que no provienen de un *peer*.

```
docker run --rm instrumentisto/nmap:7.95 -Pn -sT -p 22,80,443,3306,8000,27017,27018
IP_DEL_HOST
```

19.3 VPN

`compose.vpn.yaml` agrega un servidor WireGuard en `10.8.0.0/24` con un *peer* por integrante (`personaA` y `personaB`). Por la VPN solo se alcanzan MongoDB en `10.8.0.1:27017` y el SSH del *host* (WireGuard, s.f.). Las claves se generan en `docker/wireguard/config/`, que no se versiona. Se llena `VPN_SERVER_URL` en `.env` con la IP o el dominio del *host* y se levanta:

```
docker compose -f compose.yaml -f compose.vpn.yaml up --detach
docker exec melodic-wireguard wg show
docker exec melodic-wireguard iptables -t nat -S PREROUTING
```

La configuración de cada *peer* queda en `docker/wireguard/config/peer_personaA/peer_personaA.conf`, junto con un código QR en `.png`. Con la VPN activa en el equipo del integrante:

```
mongosh "mongodb://melodic_readonly:PASSWORD@10.8.0.1:27017/melodic?
authSource=admin&directConnection=true&tls=true&tlsCAFile=docker/mongo/tls/ca.pem"
```

El comando `wg show` debe mostrar un `latest handshake` reciente por cada *peer* conectado. El certificado de MongoDB incluye `10.8.0.1`; si se generó antes de la VPN, hay que volver a correr `generate-certs.sh` y recrear los contenedores.

20. Despliegue en línea

20.1 Servidor

Melodic está publicado en https://melodic-app.onrender.com. La página de inicio muestra el formulario de inicio de sesión de Melodic, con correo, contraseña y un enlace para registrarse.

Tabla 25

Componentes del despliegue en línea

Componente	Servicio	Configuración
Aplicación	Render, Web Service con *runtime* Docker, plan gratuito	`render.yaml` (Blueprint); despliega la rama `main`
Base de datos	MongoDB Atlas, clúster M0	`MONGODB_URI` con la cadena `mongodb+srv`
Sesiones y caché	SQLite dentro del contenedor	`DB_CONNECTION=sqlite`

20.2 Pasos

Configuración de MongoDB Atlas (MongoDB, Inc., s.f.-e):

1. Crear una cuenta en MongoDB Atlas y un clúster gratuito M0.
2. En Security > Database Access, crear el usuario `melodic_app` con permiso de lectura y escritura, de preferencia limitado a la base `melodic`, y guardar su contraseña.

3. En Security > Network Access, agregar la regla `0.0.0.0/0` para permitir la conexión desde los servidores dinámicos de Render.
4. En Connect > Drivers (PHP), copiar la cadena de conexión SRV.

```
mongodb+srv://melodic_app:<password>@<cluster-url>.mongodb.net/melodic?
retryWrites=true&w=majority
```

Despliegue en Render mediante Blueprint:

1. Conectar el repositorio de GitHub en el panel de Render.
2. Ir a Blueprints > New Blueprint Instance y seleccionar el repositorio y la rama.
3. Render lee `render.yaml` y solicita únicamente `MONGODB_URI` (la cadena de Atlas) y `APP_URL` (la URL pública asignada).

20.3 Dominio y variables de entorno

La aplicación usa el subdominio que asigna Render (`melodic-app.onrender.com`). Si el servicio se crea manualmente como Web Service con entorno Docker, se configuran estas variables:

Tabla 26

Variables de entorno en Render

Variable	Valor
`APP_NAME`, `APP_ENV`, `APP_DEBUG`	`Melodic`, `production`, `false`
`APP_KEY`	Clave generada con `php artisan key:generate --show`
`APP_URL`	`https://melodic-app.onrender.com`
`APP_LOCALE`	`es`
`DB_CONNECTION`, `DB_DATABASE`	`sqlite`, `/var/lib/melodic/database.sqlite`
`SESSION_DRIVER`, `SESSION_CONNECTION`	`database`, `sqlite`
`SESSION_SECURE_COOKIE`	`true`
`CACHE_STORE`, `DB_CACHE_CONNECTION`	`database`, `sqlite`
`QUEUE_CONNECTION`	`sync`

Variable	Valor
`MONGODB_URI`, `MONGODB_DATABASE`	Cadena `mongodb+srv` de Atlas, `melodic`

20.4 Carga de datos

Al iniciar, el contenedor ejecuta automáticamente:

1. `php artisan migrate --force` (tablas de sesiones y caché en SQLite).
2. `php artisan migrate --path=database/migrations/mongodb --force` (colecciones e índices en Atlas).
3. `php artisan db:seed --class=SubscriptionSeeder --force` (planes de suscripción, sin duplicarlos).

Para migrar datos existentes desde una base MySQL previa hacia Atlas se usan los mismos comandos del ETL: `php artisan melodic:migrate-to-mongo` y `php artisan melodic:verify-mongo`.

La imagen admite archivos de hasta 25 MB por archivo y 30 MB por petición (`docker/php/uploads.ini`) y crea el enlace `public/storage`. En el plan gratuito el disco no es persistente: las canciones y portadas subidas se pierden en cada redespliegue o reinicio.

20.5 Cuentas de acceso

Las cuentas de prueba se crean con `php artisan db:seed --force` (en local, `docker exec melodic-app php artisan db:seed --force`). En la versión en línea cada persona crea su propia cuenta, y no hay cuenta de administrador ni de sello.

Tabla 27

Cuentas de prueba del entorno local

Cuenta	Correo	Contraseña
Oyente	`user@melodic.com`	`password`

Cuenta	Correo	Contraseña
Artista	`artist@melodic.com`	`password`
Sello	`label@melodic.com`	`password`
Administrador	`admin@melodic.com`	`admin123`

20.6 Errores corregidos en producción

Tabla 28

Errores encontrados en producción (PR #18)

Error	Causa	Corrección
Un oyente nuevo no podía pasar de la página Premium	La base de Atlas no tenía planes	Los planes se siembran en cada arranque sin duplicarse
El panel de administración no hacía *scroll*	`min-h-screen` con `overflow-hidden` en el `body`	`h-screen`: el panel hace *scroll* y la barra lateral queda fija
Las canciones de más de 2 MB no se subían ni se reproducían	Límite de PHP de 2 MB y falta del enlace `public/storage`	Límite de 25 MB por archivo y enlace creado en la imagen

Los tres errores se reproducieron en local con la misma imagen antes de corregirlos.

21. Consultas y comandos en MongoDB

21.1 Conexión

Los comandos de `mongosh` toman las credenciales de las variables del contenedor, así que se pueden copiar sin editar (MongoDB, Inc., s.f.-f). Para abrir una consola como administrador o como usuario de solo lectura:

```
docker exec -it melodic-mongodb sh -c 'mongosh --tls --tlsCAFile
/run/secrets/mongodb_tls_ca -u "$MONGO_INITDB_ROOT_USERNAME" -p
"$MONGO_INITDB_ROOT_PASSWORD"'
docker exec -it melodic-mongodb sh -c 'mongosh --tls --tlsCAFile
/run/secrets/mongodb_tls_ca -u melodic_readonly -p "$MONGO_READONLY_PASSWORD" --
authenticationDatabase admin melodic'
```

Estado del *replica set* y, si no existe, su inicialización (el *healthcheck* ya lo hace):

```
docker exec melodic-mongodb sh -c 'mongosh --tls --tlsCAFile
/run/secrets/mongodb_tls_ca --quiet -u "$MONGO_INITDB_ROOT_USERNAME" -p
"$MONGO_INITDB_ROOT_PASSWORD" --authenticationDatabase admin --eval
"rs.status().members.map(m => m.name + \" \" + m.stateStr)"'
docker exec melodic-mongodb sh -c 'mongosh --tls --tlsCAFile
/run/secrets/mongodb_tls_ca --quiet -u "$MONGO_INITDB_ROOT_USERNAME" -p
"$MONGO_INITDB_ROOT_PASSWORD" --authenticationDatabase admin --eval
"rs.initiate({ _id: \"rs0\", members: [{ _id: 0, host: \"mongodb:27017\" }] })"'
```

21.2 CRUD

Las consultas se ejecutan en orden sobre los datos que carga `melodic:migrate-to-mongo`. Las referencias entre documentos son el texto del `ObjectId`, por eso se usa `.toString()` al escribirlas y `ObjectId(...)` al buscar por `_id`. Primero se definen variables de ejemplo:

```
const now = new Date();
const plan = db.subscriptions.findOne();
const artist = db.users.findOne({ role: "artist", label: null, _id: { $in:
db.albums.distinct("user_id") } });
const label = db.users.findOne({ role: "label" });
const listener = db.users.findOne({ role: "user", subscription_id: { $ne: null } });
const album = db.albums.findOne({ user_id: artist._id });
const song = db.songs.findOne({ album_id: album._id.toString() });
const playlist = db.playlists.findOne({ user_id: listener._id });
```

Ejemplo completo de CRUD sobre `subscriptions`, con su equivalente SQL:

```
// Crear · SQL: INSERT INTO subscriptions (name, cost, months) VALUES ('Anual',
999.00, 12)
const annual = db.subscriptions.insertOne({ name: "Anual", cost:
NumberDecimal("999.00"), months: 12, created_at: now, updated_at: now }).insertedId;
// Leer · SQL: SELECT * FROM subscriptions
db.subscriptions.find();
// Actualizar · SQL: UPDATE subscriptions SET cost = 899.00 WHERE id = ?
db.subscriptions.updateOne({ _id: annual }, { $set: { cost: NumberDecimal("899.00"),
updated_at: now } });
// Borrar · SQL: DELETE FROM subscriptions WHERE id = ?
db.subscriptions.deleteOne({ _id: annual });
```

Consultas de lectura sobre `users`, `songs` y `playlists`:

```
// Inicio de sesión · SQL: SELECT * FROM users WHERE email = ? LIMIT 1
db.users.findOne({ email: "user@melodic.com" });
// Últimos 10 sellos · SQL: ... WHERE role = 'label' ORDER BY created_at DESC LIMIT 10
db.users.find({ role: "label" }).sort({ created_at: -1 }).limit(10);
// Explorar · SQL: ... WHERE songs.name LIKE '%lu%' OR users.username LIKE '%lu%'
db.songs.find({ $or: [{ name: /lu/i }, { "artist.username": /lu/i }] });
// 6 al azar · SQL: SELECT * FROM songs ORDER BY RANDOM() LIMIT 6
db.songs.aggregate([{ $sample: { size: 6 } }]);
// Playlists del usuario con conteo
db.playlists.aggregate([{ $match: { user_id: listener._id } }, { $sort: { created_at:
-1 } }, { $addFields: { songs_count: { $size: "$songs" } } }]);
// Canciones de una playlist
db.songs.find({ _id: { $in: db.playlists.findOne({ _id: playlist._id }).songs.map(e =>
ObjectId(e.song_id)) } });
```

Crear, actualizar y borrar una canción con la copia del artista:

```
const track = db.songs.insertOne({ name: "Pista nueva", length: 0, plays:
NumberLong("0"), file_path: "songs/audio/pista.mp3", image_path: null, album_id:
album._id.toString(), artist: { user_id: artist._id, username: artist.username },
created_at: now, updated_at: now }).insertedId;
db.songs.updateOne({ _id: track }, { $set: { name: "Pista nueva (remix)", updated_at:
now } });
db.songs.deleteOne({ _id: track });
```

21.3 Operaciones clave con los datos reales

Reproducción con `$inc`: en SQL se hacía `UPDATE songs SET plays = plays + 1` y después se leía, sumaba y guardaba la regalía, con lo que se perdían incrementos concurrentes.

```
db.songs.updateOne({ _id: song._id }, { $inc: { plays: 1 } });
db.royalties.updateOne(
  { recipient_id: artist.label ?? artist._id },
  { $inc: { amount: NumberDecimal("0.15") }, $setOnInsert: { last_payment: null,
created_at: now }, $set: { updated_at: now } },
  { upsert: true }
);
```

Añadir una canción a una *playlist* con `$push`, sin duplicados y respetando el límite de 500:

```
db.playlists.updateOne(
  { _id: playlist._id, user_id: listener._id, "songs.song_id": { $ne:
song._id.toString() }, "songs.499": { $exists: false } },
  { $push: { songs: { song_id: song._id.toString(), added_at: now } }, $set:
{ updated_at: now } }
);
```

Retiro con `findOneAndUpdate`: se lee y se vacía el saldo en un solo paso y, con el saldo anterior, se registran el 70 % y el 30 %.

```
const before = db.royalties.findOneAndUpdate(
  { recipient_id: artist._id, amount: { $gt: NumberDecimal("0") } },
  { $set: { amount: NumberDecimal("0.00"), last_payment: now, updated_at: now } },
  { returnDocument: "before" }
);
if (before) {
  const total = Number(before.amount.toString());
  db.expenses.insertOne({ amount: NumberDecimal((total * 0.70).toFixed(2)), state:
"pagado", at: now, from: before._id.toString(), user_id: artist._id, created_at: now,
updated_at: now });
  db.earnings.insertOne({ amount: NumberDecimal((total * 0.30).toFixed(2)), state:
"pagado", at: now, subscription: null, comision: before._id.toString(), created_at:
now, updated_at: now });
}
```

21.4 Agregaciones

```
// Ingresos por mes de los últimos 6 meses
db.earnings.aggregate([
  { $match: { at: { $gte: new Date(new Date().setMonth(new Date().getMonth() - 6)) } } },
  { $group: { _id: { $dateToString: { format: "%Y-%m", date: "$at" } }, sum: { $sum: "$amount" } } },
  { $sort: { _id: 1 } }
]);
// Reproducciones totales por sello
db.users.aggregate([
  { $match: { role: "artist", label: { $ne: null } } },
  { $lookup: { from: "songs", localField: "_id", foreignField: "artist.user_id", as: "songs" } },
  { $group: { _id: "$label", plays: { $sum: { $sum: "$songs.plays" } } } }
]);
// Saldo por artista o sello
db.royalties.aggregate([
  { $lookup: { from: "users", localField: "recipient_id", foreignField: "_id", as: "recipient" } },
  { $unwind: "$recipient" },
  { $project: { _id: 0, username: "$recipient.username", role: "$recipient.role", amount: 1 } }
]);
```

La reproducción como una sola unidad requiere el *replica set* `rs0`: si falla la regalía, no se cuenta la reproducción.

```
const session = db.getMongo().startSession();
session.withTransaction(() => {
  const tx = session.getDatabase(db.getName());
  tx.songs.updateOne({ _id: song._id }, { $inc: { plays: 1 } });
  tx.royalties.updateOne(
    { recipient_id: artist.label ?? artist._id },
    { $inc: { amount: NumberDecimal("0.15") }, $set: { updated_at: new Date() } },
    { upsert: true }
  );
});
session.endSession();
```

21.5 Análisis con explain

Se ejecutó `explain("executionStats")` sobre cada consulta crítica tal como la ejecuta la aplicación, con los datos del ensayo de corte. Las escrituras se explicaron sin aplicarse.

Tabla 29

Resultados de explain de las consultas críticas

Consulta	Plan ganador	Índice	Claves	Docs	Dev.
Login por email	`EXPRESS_IXSCAN`	`email_1`	1	1	1

Consulta	Plan ganador	Índice	Claves	Docs	Dev.
Artistas de un sello	`FETCH > IXSCAN`	`label_1`	2	2	2
Últimos 10 sellos	`LIMIT > FETCH > IXSCAN`	`role_1_created_at_-1`	2	2	2
Álbumes del artista	`FETCH > IXSCAN`	`user_id_1`	2	2	2
Canciones de un álbum	`FETCH > IXSCAN`	`album_id_1`	3	3	3
Explorar sin búsqueda	`FETCH > IXSCAN`	`created_at_-1`	18	18	18
Explorar con búsqueda	`SUBPLAN > COLLSCAN`	—	0	18	3
Playlists del usuario	`FETCH > IXSCAN`	`user_id_1_created_at_-1`	6	6	6
Añadir a *playlist*	`UPDATE > FETCH > IXSCAN`	`_id_`	1	1	0
Reproducir: `$inc` plays	`EXPRESS_UPDATE`	`_id_`	1	1	0
Reproducir: `$inc` regalía	`UPDATE > FETCH > IXSCAN`	`recipient_id_1`	1	1	0
Retiro	`UPDATE > FETCH > IXSCAN`	`recipient_id_1`	1	1	0
Últimos 5 retiros	`LIMIT > FETCH > IXSCAN`	`user_id_1_at_-1`	5	5	5
Ingresos de 6 meses	`GROUP > FETCH > IXSCAN`	`at_1`	18	18	3
Total de ingresos	`GROUP > COLLSCAN`	—	0	18	1
Planes de suscripción	`COLLSCAN`	—	0	2	2

Nota. Claves = claves examinadas; Docs = documentos examinados; Dev. = documentos devueltos (en "Ingresos de 6 meses", los grupos). Se omiten algunas filas del catálogo completo de 19 consultas, disponible en `docs/mongodb/consultas-crud.md`.

Todas las consultas filtradas usan un índice y examinan solo los documentos que devuelven. Los `COLLSCAN` están justificados: la búsqueda con expresión regular, el total de ingresos (de baja frecuencia) y la lista de planes, un catálogo de 2 documentos. No faltó ningún índice, así que no hizo falta una migración nueva.

21.6 Comandos de administración

Tabla 30

Comandos de Artisan para operar Melodic

Comando	Uso
`php artisan down` / `php artisan up`	Entrar y salir de modo mantenimiento
`php artisan migrate --force`	Tablas relacionales (sesiones, caché, colas)
`php artisan migrate --path=database/migrations/mongodb --force`	Colecciones e índices de MongoDB
`php artisan melodic:migrate-to-mongo [--dry-run]`	Carga de datos (o simulación)
`php artisan melodic:verify-mongo`	Verificación contra la base relacional
`php artisan view:clear`	Borrar vistas compiladas del despliegue anterior
`php artisan db:seed --force`	Crear las cuentas de prueba

Nota. En Docker, cada comando se antepone con `docker exec melodic-app`. Después del corte, `melodic:migrate-to-mongo` sin `--dry-run` sobrescribe en MongoDB los datos con los de la base relacional, por lo que solo se usa antes del corte o en un ensayo.

Listar los índices de todas las colecciones y crear uno que falte (si ya existe con la misma definición, no hace nada):

```
docker exec melodic-mongodb sh -c 'mongosh --tls --tlsCAFile
/run/secrets/mongodb_tls_ca --quiet -u "$MONGO_INITDB_ROOT_USERNAME" -p
"$MONGO_INITDB_ROOT_PASSWORD" --authenticationDatabase admin melodic --eval
"db.getCollectionNames().forEach(c => print(c, JSON.stringify(db[c].getIndexes().map(i
=> i.key))))"'
docker exec melodic-mongodb sh -c 'mongosh --tls --tlsCAFile
/run/secrets/mongodb_tls_ca --quiet -u "$MONGO_INITDB_ROOT_USERNAME" -p
"$MONGO_INITDB_ROOT_PASSWORD" --authenticationDatabase admin melodic --eval
"db.songs.createIndex({ album_id: 1 })"'
```

Probar la conexión a MongoDB desde Laravel:

```
docker exec -e XDG_CONFIG_HOME=/tmp melodic-app php artisan tinker --execute
'dump(DB::connection("mongodb")->getClient()->listDatabaseNames());'
```

21.7 Respaldo y restauración

El respaldo se hace con `melodic_backup` (rol `backup`) a un archivo comprimido. La restauración de prueba se hace en una base temporal con el usuario *root*, porque `melodic_backup` no tiene el rol `restore`.

```
docker exec melodic-mongodb sh -c 'mongodump --quiet --uri "mongodb://melodic_backup:
$MONGO_BACKUP_PASSWORD@127.0.0.1:27017/?authSource=admin&tls=true&tlsCAFile=/run/
secrets/mongodb_tls_ca" --db melodic --archive --gzip' > melodic-$(date +
%F).archive.gz
docker exec -i melodic-mongodb sh -c 'mongorestore --quiet --uri
"mongodb://$MONGO_INITDB_ROOT_USERNAME:$MONGO_INITDB_ROOT_PASSWORD@127.0.0.1:27017/?
authSource=admin&tls=true&tlsCAFile=/run/secrets/mongodb_tls_ca" --archive --gzip --
nsFrom "melodic.*" --nsTo "melodic_restore.*" --drop' < melodic-AAAA-MM-DD.archive.gz
```

Para comparar la base original con la restaurada (conteo y *hash* por colección) y borrar la temporal:

```
docker exec melodic-mongodb sh -c 'mongosh --tls --tlsCAFile
/run/secrets/mongodb_tls_ca --quiet -u "$MONGO_INITDB_ROOT_USERNAME" -p
"$MONGO_INITDB_ROOT_PASSWORD" --authenticationDatabase admin --eval "const o =
db.getSiblingDB(\"melodic\"), r = db.getSiblingDB(\"melodic_restore\"), ho =
o.runCommand({ dbHash: 1 }).collections, hr = r.runCommand({ dbHash: 1 }).collections;
o.getCollectionNames().forEach(c => print(c, o[c].countDocuments(),
r[c].countDocuments(), ho[c] === hr[c]))"'
docker exec melodic-mongodb sh -c 'mongosh --tls --tlsCAFile
/run/secrets/mongodb_tls_ca --quiet -u "$MONGO_INITDB_ROOT_USERNAME" -p
"$MONGO_INITDB_ROOT_PASSWORD" --authenticationDatabase admin melodic_restore --eval
"db.dropDatabase()"'
```

En la prueba del 19 de octubre de 2026, con 78 documentos en 8 colecciones, `mongodump` tardó 0.1 s y generó un archivo de 4.8 KB comprimido; `mongorestore` tardó 0.9 s. Los conteos, el `dbHash` y los índices fueron iguales en las 8 colecciones.

22. Uso de la plataforma

22.1 Probar Melodic en línea

1. Entrar a https://melodic-app.onrender.com y crear una cuenta de oyente o de artista con Registrarse.
2. Como oyente, elegir un plan en Premium; después se puede explorar, escuchar y crear *playlists*.

3. Como artista, subir canciones desde el panel.

La versión en línea usa un plan gratuito: si nadie la visitó en un rato, la primera página puede tardar cerca de un minuto en abrir. Las canciones y portadas subidas se borran cada vez que el servicio se reinicia o se actualiza.

22.2 Instalación en un equipo propio

Se necesitan Docker y Git. Los comandos se escriben en una terminal, dentro de la carpeta del proyecto. Primero se descarga el proyecto y se prepara el archivo `.env` con contraseñas aleatorias:

```
git clone https://github.com/Theproxy206/Melodic-NoSQL.git
cd Melodic-NoSQL
cp .env.example .env
sed -i "s|^APP_KEY=.*|APP_KEY=base64:$(openssl rand -base64 32)|" .env
sed -i "s|^MONGO_ROOT_USERNAME=.*|MONGO_ROOT_USERNAME=root|" .env
for v in MONGO_ROOT_PASSWORD MONGO_APP_PASSWORD MONGO_READONLY_PASSWORD
MONGO_BACKUP_PASSWORD MYSQL_PASSWORD MYSQL_ROOT_PASSWORD; do
  sed -i "s|^$v=.*|$v=$(openssl rand -hex 24)|" .env
done
```

Después se generan las llaves de la base de datos (solo la primera vez), se enciende el sistema y se crean las cuentas de prueba:

```
./docker/mongo/generate-keyfile.sh
./docker/mongo/tls/generate-certs.sh
./docker/mongo/sync-secrets.sh
docker compose up --build --detach
docker compose ps
docker exec melodic-app php artisan db:seed --force
```

`docker compose ps` debe mostrar `melodic-proxy`, `melodic-app`, `melodic-mongodb` y `melodic-mysql`. Luego se abre https://localhost; el navegador avisará que el certificado no es de confianza porque es local, y se puede aceptar o confiar en él como se explica en el apartado 18.1. Para apagar el sistema se usa `docker compose down`; la información se conserva y se vuelve a encender con `docker compose up --detach`.

Para trabajar sin Docker (solo la aplicación, con SQLite):

```
composer install
cp .env.example .env
php artisan key:generate
touch database/database.sqlite
mkdir -p storage/framework/{cache/data,sessions,views} storage/logs
php artisan migrate
npm install
npm run build
composer run dev
```

Las pruebas usan MongoDB (base `melodic_testing` con el usuario `melodic_test`), así que antes hay que llenar `MONGO_TEST_PASSWORD` y `MONGODB_TEST_URI` en `.env`, levantar el *stack* con `compose.localhost.yaml` y crear el usuario como se indica en el apartado 17.1:

```
docker compose -f compose.yaml -f compose.localhost.yaml up --detach
composer test
```

Después de actualizar la rama, los cambios de `compose.yaml`, del `Dockerfile` o de `.env` se aplican así (recrea los contenedores sin borrar volúmenes):

```
docker compose up --build --detach --force-recreate --remove-orphans
docker exec melodic-app php artisan view:clear
```

22.3 Recorrido guiado por rol

Como artista

1. Entrar con `artist@melodic.com`; se abre el panel del artista.
2. Crear un álbum y subir una canción: un archivo de audio de hasta 20 MB (mp3, wav, ogg, aac o m4a) y, opcionalmente, una portada.
3. Revisar en el panel los álbumes con su número de canciones y el saldo.

Como oyente

1. Entrar con `user@melodic.com`. Como aún no tiene suscripción, el sistema lleva a la página Premium.
2. Elegir un plan para habilitar la plataforma.
3. En Explorar, buscar por nombre de canción o de artista y reproducir. Cada reproducción le paga al artista.

4. En Mis *playlists*, crear una *playlist*, abrirla y añadir canciones.

Como sello

1. Entrar con `label@melodic.com`.
2. Crear un artista del sello. Cuando sus canciones se escuchen, el saldo se acumula en la cuenta del sello.
3. Retirar el saldo: el pago aparece en los últimos retiros y el saldo vuelve a cero.

Como administrador

1. Entrar con `admin@melodic.com`.
2. Revisar el total de ingresos y egresos, la ganancia neta, la proyección y la gráfica de ingresos de los últimos seis meses.
3. Crear un sello nuevo desde el panel.

22.4 Comprobaciones

Ver cuántos usuarios y canciones hay, comprobar que la base rechaza conexiones sin cifrar (el segundo comando debe fallar) y que el sitio responde por HTTPS:

```
docker exec melodic-mongodb sh -c 'mongosh --tls --tlsCAFile
/run/secrets/mongodb_tls_ca --quiet -u melodic_readonly -p "$MONGO_READONLY_PASSWORD"
--authenticationDatabase admin melodic --eval "print(\"usuarios:\",
db.users.countDocuments(), \"canciones:\", db.songs.countDocuments())"'
docker exec melodic-mongodb sh -c 'mongosh --quiet "mongodb://127.0.0.1:27017/?
tls=false&serverSelectionTimeoutMS=3000" --eval "1"'
curl -sI -k https://localhost | head -1
curl -sI http://localhost | head -2
```

22.5 Problemas comunes

Tabla 31

Problemas comunes y su solución

Qué pasa	Qué hacer
`docker compose up` indica que falta una variable (`... is required`)	Falta una contraseña en `.env`; repetir su preparación (apartado 22.2)

Qué pasa	Qué hacer
`docker compose up` indica `external volume "melodic-nosql_mongodb-secrets" not found`	Ejecutar `./docker/mongo/sync-secrets.sh`
El navegador muestra "conexión no segura"	Es el certificado local; aceptar o confiar en la CA (apartado 18.1)
Una canción no sube	El archivo debe pesar como máximo 20 MB y ser mp3, wav, ogg, aac o m4a
La página en línea tarda en abrir	El plan gratuito se suspende sin visitas; esperar cerca de un minuto
Aparece "demasiados intentos" al iniciar sesión	Solo se permiten 5 intentos por minuto; esperar un minuto
Tras actualizar el proyecto, la página se ve igual o da error	Ejecutar `docker exec melodic-app php artisan view:clear`
El oyente siempre termina en la página Premium	No tiene suscripción activa: elegir un plan

23. Pruebas de control de acceso

23.1 Usuario de solo lectura

El usuario `melodic_readonly` puede consultar la base `melodic`, pero un intento de inserción devuelve `not authorized`. Lo mismo ocurre con `melodic_backup`, cuyo rol solo permite respaldar.

23.2 Usuario de la aplicación

El usuario `melodic_app` puede escribir en `melodic`, pero no puede borrar la base `admin` ni leer `config` u otras bases:

```
docker exec melodic-mongodb sh -c 'mongosh --tls --tlsCAFile
/run/secrets/mongodb_tls_ca --quiet -u melodic_app -p "$MONGO_APP_PASSWORD" --
authenticationDatabase admin --eval "db.getSiblingDB(\"admin\").dropDatabase()"'
```

Tabla 32

Pruebas de permisos en MongoDB

Usuario	Prueba	Resultado
`melodic_app`	`dropDatabase` en `admin`	`not authorized`
`melodic_app`	Leer `config` u otra base	`not authorized`
`melodic_app`	Escribir en `melodic`	Permitido
`melodic_readonly`	Insertar en `melodic`	`not authorized`

Usuario	Prueba	Resultado
`melodic_backup`	Insertar en `melodic`	`not authorized`

23.3 Sin credenciales y sin cifrado

Una conexión sin TLS es rechazada por `requireTLS`, y una conexión sin credenciales no puede ejecutar operaciones sobre las bases, porque la autenticación está activada con *keyFile*:

```
docker exec melodic-mongodb sh -c 'mongosh --quiet "mongodb://127.0.0.1:27017/?
tls=false&serverSelectionTimeoutMS=3000" --eval "db.runCommand({ ping: 1 })"'
```

23.4 Verificación automática en la aplicación

Las 42 pruebas funcionales de `composer test` comprueban, entre otras cosas, que:

- El registro rechaza los roles `admin` y `label`.
- Un oyente recibe 403 en el panel del artista y quien no es administrador, en el panel de administración.
- Solo el dueño puede ver o modificar su *playlist*.
- Un oyente sin suscripción es enviado a la página Premium.

24. Limitaciones y trabajo futuro

Limitaciones identificadas:

- El *replica set* tiene un solo nodo: permite transacciones, pero no alta disponibilidad.
- El *firewall* se probó en un contenedor aislado; aplicarlo en el *host* requiere `sudo` y queda como paso manual.
- En local, los certificados son de CA propias.
- El plan gratuito de Render se suspende sin visitas, no tiene disco persistente y las sesiones viven en SQLite dentro del contenedor (se cierran al reiniciar).
- En producción, Atlas acepta conexiones desde cualquier IP, no se envía la cabecera HSTS y no hay cuenta de administrador ni de sello.

- La vista de la *playlist* no muestra los mensajes de la aplicación (por ejemplo, que una canción ya estaba o que la *playlist* está llena).
- La búsqueda por texto usa expresiones regulares y recorre la colección completa.
- No hay validadores `$jsonSchema` y las pruebas necesitan un MongoDB en ejecución.

Trabajo futuro:

- *Replica set* de tres nodos y respaldos programados.
- Validadores `$jsonSchema` por colección.
- Atlas Search o un índice de texto para la búsqueda.
- Escribir `plays` para estadísticas por oyente y retirar `legacy_id` al terminar la transición.
- Llevar sesiones y caché a Redis y usar almacenamiento externo (como S3) para los archivos subidos.
- Un *middleware* que agregue HSTS en producción, una lista de IP fijas para Atlas y una cuenta de administrador en producción con contraseña tomada del entorno.

25. Conclusión

La migración se completó en un *sprint* con las 28 tareas cerradas. Melodic funciona sobre MongoDB con un modelo diseñado a partir de sus consultas: las *playlists* embebidas, la copia del artista y el saldo en su propia colección resuelven en un solo documento las operaciones más frecuentes, y cada consulta crítica tiene un índice comprobado con `explain()`.

El cambio de base de datos sirvió además para corregir defectos del sistema original: reproducir y retirar ahora son operaciones atómicas, el dinero se calcula sin errores de redondeo y el panel de administración funciona en cualquier entorno. La carga es repetible y verificable, y el corte se ensayó dos veces junto con una reversión de 31 segundos.

El sistema quedó publicado en Render con MongoDB Atlas, y los errores que solo aparecieron en producción se reprodujeron en local con la misma imagen antes de corregirlos. En seguridad, el sistema pasó de tener secretos en el repositorio y puertos abiertos a usar cifrado en todas sus conexiones, usuarios con permisos mínimos, una red aislada, un *firewall* que cubre los puertos de Docker y acceso administrativo solo por VPN.

Las decisiones que cambiaron respecto al plan (referencias como texto, sin copia del álbum y corte por modelos) surgieron de probar el diseño contra el código real, y quedaron documentadas junto con su motivo.

Referencias

Caddy. (s.f.). *Caddy documentation*. https://caddyserver.com/docs/

Docker, Inc. (s.f.). *Packet filtering and firewalls*. Docker Docs. https://docs.docker.com/engine/network/packet-filtering-firewalls/

Laravel. (s.f.). *Laravel 12.x documentation*. https://laravel.com/docs/12.x

MongoDB, Inc. (s.f.-a). *Transactions*. MongoDB Manual. https://www.mongodb.com/docs/manual/core/transactions/

MongoDB, Inc. (s.f.-b). *Data modeling*. MongoDB Manual. https://www.mongodb.com/docs/manual/data-modeling/

MongoDB, Inc. (s.f.-c). *Laravel MongoDB*. MongoDB Docs. https://www.mongodb.com/docs/drivers/php/laravel-mongodb/current/

MongoDB, Inc. (s.f.-d). *Security*. MongoDB Manual. https://www.mongodb.com/docs/manual/security/

MongoDB, Inc. (s.f.-e). *MongoDB Atlas documentation*. https://www.mongodb.com/docs/atlas/

MongoDB, Inc. (s.f.-f). *MongoDB Shell (mongosh)*. https://www.mongodb.com/docs/mongodb-shell/

Morales Villarauz, L. D. y Soberano Paredes, S. E. (2026a). *Melodic-NoSQL* [Repositorio de código fuente]. GitHub. https://github.com/Theproxy206/Melodic-NoSQL

Morales Villarauz, L. D. y Soberano Paredes, S. E. (2026b). *Melodic* [Aplicación web]. Render. https://melodic-app.onrender.com/

Render. (s.f.). *Render documentation*. https://render.com/docs

WireGuard. (s.f.). *WireGuard: Fast, modern, secure VPN tunnel*. https://www.wireguard.com/

Anexo. Repositorio y archivos principales

Tabla 33

Enlaces del proyecto

Recurso	Enlace
Repositorio principal	https://github.com/Theproxy206/Melodic-NoSQL
Ramas del repositorio	https://github.com/Theproxy206/Melodic-NoSQL/branches
Aplicación en línea	https://melodic-app.onrender.com/

Tabla 34

Documentación del repositorio

Documento	Contenido
`docs/scrum.md`	Plan del *sprint* y tarjetas
`docs/manual-usuario.md`	Manual de usuario
`docs/manual.md`	Instalación y configuración
`docs/informe-migracion-nosql.md`	Informe técnico de la migración
`docs/mongodb/01-inventario.md`	Modelo relacional y catálogo de consultas
`docs/mongodb/02-diseno.md`	Colecciones, tipos e índices
`docs/mongodb/03-corte-y-reversion.md`	Procedimiento y bitácora del corte
`docs/mongodb/consultas-crud.md`	Consultas CRUD, agregaciones y *explain*
`docs/mongodb/hardening.md`	Evidencia de TLS y puertos
`docs/mongodb/comandos.md`	*Runbook* de operación
`docs/despliegue-render-atlas.md`	Despliegue en Render con MongoDB Atlas
`docs/migracion-mongodb/FASE-0-LINEA-BASE.md`	Línea base inicial

Tabla 35

Archivos principales del código

Archivo	Función
`compose.yaml`	Servicios `proxy`, `app`, `mongodb` y `mysql`
`compose.localhost.yaml`	Publica MongoDB solo en `127.0.0.1:27018` para desarrollo

Archivo	Función
`compose.vpn.yaml`	Servidor WireGuard opcional
`render.yaml`	Blueprint del despliegue en Render
`docker/mongo/generate-keyfile.sh`, `tls/generate-certs.sh`, `sync-secrets.sh`	*KeyFile*, certificados y volumen de secretos
`docker/firewall/ufw-setup.sh`	Reglas de ufw y `DOCKER-USER`
`docker/php/uploads.ini`	Límites de subida de archivos
`database/migrations/mongodb/`	Colecciones e índices de MongoDB